```cpp
#include <iostream>
#include <algorithm>
#include <stdexcept>  // 用于 std::out_of_range
#include <string>

class Dynarray {
private:
    std::size_t _size;
    int* _data;

    void swap(Dynarray& h) noexcept {
        std::swap(_size, h._size);
        std::swap(_data, h._data);
    }

public:
    using size_type = std::size_t;
    using value_type = int;
    using pointer = int*;
    using reference = int&;
    using const_pointer = const int*;
    using const_reference = const int&;

    Dynarray() : _size(0), _data(nullptr) {}

    explicit Dynarray(size_type n) : _size(n), _data(new value_type[n]{}) {}

    Dynarray(size_type n, value_type x) : _size(n), _data(new value_type[n]) {
        std::fill(_data, _data + _size, x);
    }

    Dynarray(const const_pointer begin, const const_pointer end)
        : _size(end - begin), _data(new value_type[_size]) {
        std::copy(begin, end, _data);
    }

    Dynarray(const Dynarray& other) : _size(other._size), _data(new value_type[_size]) {
        std::copy(other._data, other._data + _size, _data);
    }

    Dynarray(Dynarray&& other) noexcept : _size(other._size), _data(other._data) {
        other._data = nullptr;
        other._size = 0;
    }

    ~Dynarray() {
        delete[] _data;
    }

    Dynarray& operator=(const Dynarray& p) {
        if (this != &p) {
            Dynarray temp(p); // 临时对象 p的副本
            swap(temp);  // 交换后临时对象temp会自动析构
        }
        return *this;
    }

    Dynarray& operator=(Dynarray&& other) noexcept {
        if (this != &other) {
            delete[] _data;
            _data = other._data;
            _size = other._size;
            other._data = nullptr;
            other._size = 0;
        }
        return *this;
    }

    // 提供公共接口访问 _data
    const_pointer data() const {
        return _data;
    }

    reference operator[](size_type i) {
        return _data[i];
    }

    const_reference operator[](size_type i) const {
        return _data[i];
    }

    size_type size() const noexcept {
        return _size;
    }

    bool empty() const noexcept {
        return _size == 0;
    }

    int& at(size_type n) {
        if (n >= _size) {
            throw std::out_of_range("Dynarray index out of range!");
        }
        return _data[n];
    }

    const int& at(size_type n) const {
        if (n >= _size) {
            throw std::out_of_range("Dynarray index out of range!");
        }
        return _data[n];
    }

    // 输出字符串 e.g [1, 2, 3, 5]
    std::string output() const {
        std::string a = "[";
        for (size_t i = 0; i < _size; ++i) {
            a += std::to_string(_data[i]); // 将int转为string
            if(i != _size - 1){
                a += ", ";
            }
        }
        a += "]";
        return a;
    }
};

// 全局比较操作符 在类外，有两个参数
bool operator==(const Dynarray& lhs, const Dynarray& rhs) {
    return std::equal(lhs.data(), lhs.data() + lhs.size(), rhs.data(), rhs.data() + rhs.size());
}

bool operator<(const Dynarray& lhs, const Dynarray& rhs) {
    return std::lexicographical_compare(lhs.data(), lhs.data() + lhs.size(), rhs.data(), rhs.data() + rhs.size()
}

bool operator!=(const Dynarray& lhs, const Dynarray& rhs) {
    return !(lhs == rhs);
}

bool operator>(const Dynarray& lhs, const Dynarray& rhs) {
    return rhs < lhs;
}

bool operator<=(const Dynarray& lhs, const Dynarray& rhs) {
    return !(lhs > rhs);
}

bool operator>=(const Dynarray& lhs, const Dynarray& rhs) {
    return !(lhs < rhs);
}

std::ostream& operator<<(std::ostream& os, const Dynarray& arr) {
    os << arr.output();
    return os;
}
```

4. (43 points) Sorting Algorithm

The following code presents the initial definition of the Student class. The numbered positions *(1)*, *(2)*, *(3)*, etc., indicate areas where modifications or questions may arise:

```cpp
class Student {
private:
  std::string m_name;
  int m_age;

public:
  Student() = delete;
  ~Student() = default;

  Student(std::string name, int age): m_name(std::move(name)), m_age(age) {} //(1)

  const std::string& getName() const { return m_name; }

  void setAge(int age) { m_age = age; }

  int getAge() const { return m_age; }
  //(2)
  //(3)
};
//(4)
//(5)
```

(1) (10') Compare the following four constructor initializer list scenarios:

1. `Student(const std::string &name, ... ) : m_name(name), ...`
2. `Student(const std::string &name, ... ) : m_name(std::move(name)), ...`
3. `Student(std::string name, ... ) : m_name(name), ...`
4. `Student(std::string name, ... ) : m_name(std::move(name)), ...`

Identify which combination of input parameter types and operations (whether `std::move` is used or not) is the most efficient, and provide a detailed explanation for your reasoning

Solution:

1. `Student(const std::string &name, ... ) : m_name(name), ...`

   Lvalue: Binds, then copies. **Rvalue:** Moves, then copies (inefficient). **[1 + 1 points]**

2. `Student(const std::string &name, ... ) : m_name(std::move(name)), ...`

   Lvalue: Binds, then copies. **Rvalue:** Moves, then copies (inefficient). **[1 + 1 points]**

   Explanation:

   `std::move()` can be used on a `const` lvalue, but it will **NOT** enable the object to be moved. This is because the move constructor of `std::string` accepts an rvalue reference to a non-const object. When applied to a const lvalue, `std::move()` results in a `const &&`, which cannot be passed to the move constructor. As a result, the object is copied rather than moved.

3. `Student(std::string name, ... ) : m_name(name), ...`

   Lvalue: Copies twice (inefficient). **Rvalue:** Moves, then copies (inefficient). **[1 + 1 points]**

4. `Student(std::string name, ... ) : m_name(std::move(name)), ...`

   The most efficient constructor for both lvalues and rvalues. **[2 point]**

   Lvalue: Copies, then moves (efficient). **Rvalue:** Moves twice (efficient). **[1 + 1 points]**

(2) (4') The following code demonstrates an attempt to directly output `Student` objects via `std::cout`.

```cpp
#include <iostream>
#include <vector>

int main() {
  std::vector<Student> students = {{"Bob", 17}, {"Alice", 23}, {"John", 19}};

  for (const auto &student : students) {
    std::cout << student;
  }
}
```

Implement an **overload** for the `operator<<` to enable the output of **ALL** data members of the `Student` class when a `Student` object is passed to `std::cout`.

Solution:

Note that the 'getters', `getName()` and `getAge()`, have been provided for you. Therefore, it is **not necessary** to declare the function as a `friend`.

```cpp
// At (4)
std::ostream& operator<<(std::ostream &os, const Student &student) {
  return os << "Student " << student.getName()
            << "'s age is: " << student.getAge() << std::endl;
}
```

Alternatively, if you choose to forgo the use of the provided `getters`, there is no penalty for declaring the function as `friend` in order to access the private data members, `m_name` and `m_age`, directly.

```cpp
// At (2)
friend std::ostream& operator<<(std::ostream &os, const Student &student);
```

```cpp
// At (4)
std::ostream& operator<<(std::ostream &os, const Student &student) {
  return os << "Student " << student.m_name
            << "'s age is: " << student.m_age << std::endl;
}
```

Note: Any other valid implementation is considered correct. The grading breakdown is as follows:

- **2 points** for a correct function declaration (parameters, return values),
- **2 points** for returning the passed-in reference `os`.

```
error: no match for 'operator<' (operand types are 'Student' and 'Student')
```

```cpp
// At (3)
bool operator<(const Student &other) const {
  return m_age < other.m_age;  // using the getter getAge() is also correct
}
```

Alternatively, you can define it as a non-member function:

```cpp
// At (5)
bool operator<(const Student &lhs, const Student &rhs) {
  return lhs.getAge() < rhs.getAge();
}
```

ii. (6') **Method 2: Using a Lambda Expression**

Without overloading the `<` operator, it is still possible to enable the usage of `std::sort`. Implement a lambda expression with `std::sort` to sort the `students` vector in ascending order by age. Modify the line marked with an asterisk (*) in the following code snippet:

Solution:

```cpp
std::sort(students.begin(), students.end(), []
    (const Student& lhs, const Student& rhs)
    -> bool { return lhs.getAge() < rhs.getAge(); });
```

Here is some incomplete code of these classes. Complete it according to the instructions.

```
class Time {
 public:
  Time(int minutes = 0)
    : m_hour((minute / 60) % 24), m_minute(minute % 60) {}
  Time(int hour, int minute)
    : m_hour((hour + minute / 60) % 24), m_minute(minute % 60) {}
  // (1)
 private:
  int m_hour;
  int m_minute;
  static std::string fill2(int x) {
    return x < 10 ? "0" + std::to_string(x) : std::to_string(x);
  }
};
class Clock {
 public:
  Clock(Time time) : m_time(time) {}
  void tick() {
    ++m_time; // It should add 1 minute to m_time.
  }
  virtual void display() const {
    // It should print something like "Now: 09:03".
    std::cout << "Now: " <<  m_time << std::endl;
  }
  // (*)
 protected:
  Time m_time;
};
class AlarmClock : public Clock {
 public:
  AlarmClock(Time time, Time alarm)
    : Clock(time), m_alarm(alarm) {}
  // (2)
 private:
  Time m_alarm;
};
// (3)
```

(1) (3') `Clock` should have a destructor at line (*). Write the destructor so that no undefined behavior happens when `Clock` and `AlarmClock` are used in polymorphism.

Solution:

```
virtual ~Clock() = default;
```

Also correct:

```
virtual ~Clock() {}
```

(2) (6') Currently the code does not compile, because `Time` lacks some operators. Add code to (1), (3), or both places, so that the code compiles and behaves like stated in the comments. Specify which of your code goes to (1) or (3). You may find the `Time::fill2` function useful.

Solution: (1):

```
Time &operator++() {
  ++m_minute;
  if (m_minute == 60) {
    m_hour = (m_hour + 1) % 24;
    m_minute = 0;
  }
  return *this;
}
friend std::ostream &operator<<(std::ostream &, const Time &);
```

(3):

```
friend std::ostream &operator<<(std::ostream &os, const Time &t) {
  return os << Time::fill2(t.m_hour) << ":" << Time::fill2(t.m_minute);
}
```

This friend function `operator<<` can also be defined at (1). `Time::fill2` can be replaced with `t.fill2`.

(3) (6') Finally, `AlarmClock` should override `display()`, in the following manner:

- It should print 2 or 3 lines.
- The first line should look like `Now: 09:03`, identical to what `Clock::display()` prints.
- The second line should look like `Alarm: 09:15`, printing the value of `m_alarm`.
- If the time now is its alarm time, print `ALARM!` on the third line.

Add code to do so. Feel free to add any other functions. Any of your code can go to (1), (2), or (3), and you should specify where it goes.

Solution: (1):

```
bool operator==(const Time &rhs) const {
  return m_hour == rhs.m_hour && m_minute == rhs.m_minute;
}
```

(2):

```
void display() const override {
  Clock::display();
  std::cout << "Alarm: " << m_alarm << std::endl;
  if (m_alarm == m_time)
    std::cout << "ALARM!" << std::endl;
}
```

Other ways to enable `AlarmClock::display` to compare `m_alarm` with `m_time` are allowed. For example, declare `AlarmClock` as a friend class of `Time` in (1), so that `m_hour` and `m_minute` can be accessed directly.

The first line can also be printed directly using

```
std::cout << "Now: " << m_time << std::endl;
```

2. 需要 `::` 的情况

如果派生类**覆盖了基类的函数**，但想显式调用基类的版本，则需要通过 `Base::` 指定作用域。

cpp 复制 下载

```
class Base {
public:
    void foo() { cout << "Base::foo" << endl; }
};

class Derived : public Base {
public:
    void foo() {
        Base::foo(); // 显式调用基类版本
        cout << "Derived::foo" << endl;
    }
};

int main() {
    Derived d;
    d.foo();      // 调用 Derived::foo()
    d.Base::foo(); // 显式调用 Base::foo()
}
```

```
int main() {
    MyClass obj1(10, "test");
    MyClass obj2;
    obj2 = obj1;  // 调用拷贝赋值运算符
    return 0;
}
```

```
int main() {
    MyClass obj1(10, "test");
    MyClass obj2 = obj1;  // 调用拷贝构造函数
    useObject(obj1);      // 调用拷贝构造函数
    return 0;
}
```

```
    // 拷贝赋值运算符
    ResourceHolder& operator=(const ResourceHolder& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = size ? new int[size] : nullptr;
            std::copy(other.data, other.data + size, data);
        }
        return *this;
    }

    // 移动赋值运算符
    ResourceHolder& operator=(ResourceHolder&& other) noexcept {
        if (this != &other) {
            delete[] data;
            data = other.data;
            size = other.size;
            other.data = nullptr;
            other.size = 0;
        }
        return *this;
    }
};
```

```
class ResourceHolder {
private:
    int* data;
    size_t size;

public:
    // 构造函数
    explicit ResourceHolder(size_t s = 0) : size(s), data(s ? new int[s] : nullptr) {}

    // 析构函数
    ~ResourceHolder() {
        delete[] data;
    }

    // 拷贝构造函数
    ResourceHolder(const ResourceHolder& other) : size(other.size), data(other.size ? new int[ot
her.size] : nullptr) {
        std::copy(other.data, other.data + other.size, data);
    }

    // 移动构造函数
    ResourceHolder(ResourceHolder&& other) noexcept : data(other.data), size(other.size) {
        other.data = nullptr;
        other.size = 0;
    }
```

const menber functions
不能对members进行修改操作、不能调用非const的成员函数
在class X的const成员函数中，this的类型是const X *,非const成员函数中,this的类型是X *

```cpp
#include <iostream>
#include <utility>  // for std::move

// 用于检测左右值的重载函数
void check_value(int&) { std::cout << "lvalue\n"; }
void check_value(int&&) { std::cout << "rvalue\n"; }

int main() {
    // 1. 基本变量
    int x = 10;           // x是左值
    check_value(x);       // lvalue
    check_value(20);      // rvalue
    check_value(x + 5);   // rvalue

    // 2. 指针相关
    int* p = &x;          // p是左值
    check_value(*p);      // lvalue (解引用)
    check_value(p[0]);    // lvalue (数组下标)
    int* p2 = p + 1;      // p+1是右值
    // check_value(p + 1); // 错误：不能将指针转换为int

    // 3. 左值引用
    int& r = x;           // r是左值引用(绑定到左值)
    check_value(r);       // lvalue
    // int& r2 = 20;      // 错误：不能绑定右值到非const左值引用

    // 4. const左值引用
    const int& cr1 = x;   // 绑定左值OK
    const int& cr2 = 30;  // 绑定右值OK

    // 5. 右值引用
    int&& rr = 40;        // rr是右值引用(绑定到右值)
    check_value(rr);      // lvalue (rr有名字，所以是左值)
    check_value(std::move(rr)); // rvalue (使用std::move)

    // 6. 函数返回值
    auto getInt = []() { return 50; };
    check_value(getInt()); // rvalue
    auto getRef = [&x]() -> int& { return x; };
    check_value(getRef()); // lvalue

    // 7. 指针的引用
    int*& rp = p;         // 对指针的左值引用
    check_value(*rp);     // lvalue
    int*&& rrp = std::move(p); // 对指针的右值引用
    check_value(*rrp);    // lvalue

    // 8. 类型推导中的引用折叠
    auto&& a1 = x;        // int& (左值引用)
    auto&& a2 = r;        // int& (左值引用)
    auto&& a3 = rr;       // int& (左值引用)
    auto&& a4 = 60;       // int&& (右值引用)

    std::cout << "a1 is lvalue? "; check_value(a1);  // lvalue
    std::cout << "a4 is lvalue? "; check_value(a4);  // lvalue
```

6. (20 points) For each piece of code, write down the type of var.

(a) `double* a, b[10], var;`

(a) double

(b) `void foo(int var[3][4]) { /* ... */ }`

(b) int (*)[4]

(c) `int var[3][4], *x = &var[0][0];`

(c) int [3][4]

(d) `char *var[] = {"hello", "world"};`

(d) char *[2]

3. (10 points) Design a function `swap` that can swap two pointers (both of type `int *`).

```
int i = 10, j = 15, *p1 = &i, *p2 = &j;
swap(/* you design this */);
```

After this call to `swap`, `p1` should be pointing to `j` and `p2` should be pointing to `i`.

A. `void swap(int *p1, int *p2) { int *tmp = p1; p1 = p2; p2 = tmp; }`
To call this function: `swap(p1, p2);`

B. `void swap(int **p1, int **p2) { int *tmp = *p1; *p1 = *p2; *p2 = tmp; }`
To call this function: `swap(&p1, &p2);`

C. `void swap(int **p1, int **p2) { int **tmp = p1; p1 = p2; p2 = tmp; }`
To call this function: `swap(&p1, &p2);`

D. `void swap(int *p1, int *p2) { int **tmp = &p1; &p1 = &p2; &p2 = tmp; }`
To call this function: `swap(*&p1, *&p2);`

方法4：使用C++引用（如果是C++环境）

cpp 复制 下载

```cpp
void swap(int* &a, int* &b) {
    int *temp = a;
    a = b;
    b = temp;
}
```

使用方式：

cpp 复制 下载

```cpp
swap(p1, p2);
```

```
class BinaryOp : public Expression {
protected:
  Node left, right;
public:
  BinaryOp(Node l, Node r) : Expression{}, left{std::move(l)}, right{std::move(r)} {}
  // ...
};
class PlusOp : public BinaryOp {
  using BinaryOp::BinaryOp;
  // ...
};
```

By declaring `using BinaryOp::BinaryOp;`, the class `PlusOp` has a constructor inherited from `BinaryOp` which has the similar effect as

```
PlusOp(Node l, Node r) : BinaryOp(std::move(l), std::move(r)) {}
```

that is, it accepts the same parameters and initializes the base class subobject in the same way as the base class's constructor. Based on your understanding about classes and inheritance, answer the following questions.

(a) (10 points) The `using` declaration is written in a `class`, without an explicitly written access specifier. What is the access level of the inherited constructor (public, private, or protected)? Explain your idea.

(b) (10 points) If `PlusOp` also has its own data members, how are they initialized in the inherited constructor?

Solution:

(a) The access level of the inherited constructor is the same as in the base class, so the inherited constructor of `PlusOp` is public. Inheritance shall not break the encapsulation of the base class.

(b) They are initialized in declaration order after the initialization of the base class subobject. For each data member,

- if it has a in-class initializer, it is initialized from that initializer;
- otherwise, it is default-initialized. If it is not default-initializable, the program is ill-formed.

引用法传参：避免传参时发生的copy操作(不引用则函数通过copy传参)
exception：引用只可绑定左值，但const reference可绑定右值。如右值传参

```
const std::string &rcs = a + b; //it`s OK
int count_lowercase(std::string &str) { // `str` is a reference.}
int result = count_lowercase(s1 + s2);
```

此时右值a+b先转化成左值tmp,rcs再对左值tmp进行引用绑定。右转左过程中并不发生copy，而是发生move（assignment move）→ move is also cheap！

```
struct Ghost {
  int posX;
  int posY;
  char icon;
  bool isSmart;
};
```

allocate a block of memory to store `n` ghosts: `malloc( n * sizeof(struct Ghost) )`.

(5) (15') [C++] Suppose `s` is an object of type `std::string`. Use `new` to create a `std::string` that is move-constructed from `s`:

`auto t = new std::string(std::move(s)) ;`

The type of `t` is `std::string *`.

To destroy this string and deallocate the memory: `delete t;`

(6) (15') The following code reads `n` strings from input, and stores them into a `std::vector<std::string>`. Fill in the blank with **the best way** of appending the string `s` to the end of `strings`.

```
std::vector<std::string> strings;
for (auto i = 0; i != n; ++i) {
  std::string s;
  std::cin >> s;
  strings .push_back(std::move(s)) ;
}
```

Suppose `k` is a variable of type `std::size_t`. Use a *range-based `for` loop* to traverse `strings` to count how many of the strings have length **less than k**. Fill in the blank with **the best answer**.

```
int cnt = 0;
for ( const auto &s : strings ) {
  if (s.size() < k)
```

`std::count_if` is a standard library function that accepts **a sequence** and **a unary predicate**, and returns the number of elements in the sequence for which the predicate returns `true`. Use `std::count_if` to rewrite the code above.

```
int cnt = std::count_if(
    strings.begin()                    ,
    strings.end()                      ,
    [k](const std::string &s) { return s.size() < k; }
);
```

```
class Dynarray {
public:
    int *       begin()       { return m_storage; }
  const int *   begin()  const { return m_storage; }
                begin()
    int *       end()         { return m_storage + m_length; }
  const int *   end()    const { return m_storage + m_length; }
                end()
```

Solution: One possible way: a `protected` non-`virtual` function.

```
class DialogueBox {
  std::string caption;
  // ...

public:
  virtual void display() = 0;

protected:
  void defaultDisplay() {
    // DB
  }
};
```

```
class Progress : public DialogueBox {
public:
  void display() override {
    defaultDisplay();
  }
};
```

Another way: a pure `virtual` function with a definition.

```
class DialogueBox {
  std::string caption;
  // ...

public:
  virtual void display() = 0;
};

void DialogueBox::display() {
  // DB
}
```

```
class Progress : public DialogueBox {
public:
  void display() override {
    DialogueBox::display();
  }
};
```

3. (30 points) Given the following classes `Item` and `DiscountedItem`

```
class Item {
protected:
  std::string mName;
  double mPrice;
public:
  Item(std::string name, double price) : mName{std::move(name)}, mPrice{price} {}
  virtual double netPrice(unsigned cnt) const { return mPrice * cnt; }
  virtual ~Item() = default;
};
class DiscountedItem : public Item {
protected:
  unsigned mMinQuantity;
  double mDiscount;
public:
  DiscountedItem(std::string name, double price, unsigned minQuantity, double discount)
      : Item{std::move(name), price}, mMinQuantity{minQuantity}, mDiscount{discount} {}
  double netPrice(unsigned cnt) const override {
    return cnt < mMinQuantity
           ? mPrice * cnt
           : mPrice * mMinQuantity + mPrice * mDiscount * (cnt - mMinQuantity);
  }
};
```

Define a class `LimitedDiscountItem` (you can write it as `LDI`) that implements a *limited discount strategy*, which applies a discount to items purchased up to a given limit. If the number of items purchased exceeds that limit, the normal price applies to those purchased beyond the limit.

Will you make `LimitedDiscountItem` inherit from `Item`? `DiscountedItem`? Or, design a completely different inheritance hierarchy? Write your code and explain your design on this. Your code should include at least the data members of `LimitedDiscountItem` and its `netPrice` function.

Solution: We may define `LimitedDiscountItem` as a derived class of `Item`, because a `LimitedDiscountItem` is an `Item`: Wherever an `Item` is required, a `LimitedDiscountItem` also works.

```
class LimitedDiscountItem : public Item {
  double mDiscount;
  unsigned mMaxQuantity;
public:
  LimitedDiscountItem(std::string name, double price, double discount, unsigned maxQuantity)
      : Item{std::move(name), price}, mDiscount{discount}, mMaxQuantity{maxQuantity} {}
  double netPrice(unsigned cnt) const override {
    return cnt <= mMaxQuantity
           ? mPrice * mDiscount * cnt
           : mPrice * mDiscount * mMaxQuantity + mPrice * (cnt - mMaxQuantity);
  }
};
```

Writing another base class and letting `Item`, `DiscountedItem` and `LimitedDiscountItem` inherit from them is also a correct design, but not necessary and has no obvious benefits.

```
class Expression {
public:
  / (2 points) Expression() = default;
    virtual int evaluate() const = 0; (3 points)
  virtual std::string toString() const = 0; (3 points)
    virtual ~Expression() = default; (3 points)
                                                    
                                                    
};
```

```
class Dynarray {
  int *m_storage;
  std::size_t m_length;
public:
  Dynarray sorted() const {
    Dynarray ret = *this;
    std::sort(ret.m_storage, ret.m_storage + ret.m_length);
    return ret;
  }
  // some other members ...
};
```

The member function `sorted()` returns a copy of `*this` but with all elements sorted in ascending order. However, if `sorted()` is called on a non-`const` rvalue (e.g. `std::move(a).sorted()`, or `Dynarray(begin, end).sorted()`), there is no need to copy the original array - we can directly sort the elements in `*this`, and return an rvalue reference to `*this` (obtained by `std::move`).

Use the `const`- and ref-qualifications of member functions to achieve this. Fill in the blanks for the return types and qualifications, and complete the function bodies.

Solution:

```
class Dynarray {
  int *m_storage;
  std::size_t m_length;
public:
  Dynarray sorted() const & {
    auto ret = *this;
    std::sort(ret.m_storage, ret.m_storage + ret.m_length);
    return ret;
  }
  Dynarray &&sorted() && {
    std::sort(m_storage, m_storage + m_length);
    return std::move(*this);
  }
  // some other members ...
};
```

The return type of the second function was required to be `Dynarray &&` (which yields an **xvalue**), but it is also ok to use `Dynarray` (which yields a **prvalue**). But the returned expression must be `std::move(*this)`.

5. 静态存储期变量（初始化为0）

- **规则**：全局变量、`static` 变量或 `thread_local` 变量会被零初始化。
- **示例**：

```cpp
int global[5]; // 全部初始化为0（静态存储期）
void foo() {
    static int arr[5]; // 全部初始化为0
}
```

总结表格

场景	是否初始化为0	示例
默认初始化（局部变量）	❌ 否	`int arr[5];`
值初始化（`{}` 或 `()`）	✅ 是	`int arr[5]{};`
显式部分初始化	✅ 未指定部分为0	`int arr[5] = {1};`
动态分配（无 `()`）	❌ 否	`new int[5];`
动态分配（带 `()`）	✅ 是	`new int[5]();`
`std::vector`	✅ 是	`std::vector<int> v(5);`
静态存储期变量	✅ 是	`static int arr[5];`

```cpp
bool List::contains(int val) {
  if (!head)
    return false; // The list is empty.

  if (head->value == val)
    return true; // The value is already at the front.

  // Define two raw pointers to traverse the list.
  Node *prev = head.get();
  Node *current = head->next.get();

  while (current) {
    if (current->value == val) { // Now we have found the val.
      auto newHead = std::move(prev->next);
      prev->next = std::move(newHead->next);
      newHead->next = std::move(head);
      head = std::move(newHead);
      return true;

    }
    prev = current;
    current = prev->next.get();
  }
  return false; // No matched val.
}
```

(3) (7') The task is to implement the `push_front` function, which inserts a new node at the head of the linked list. Please complete the code segments as indicated below. Note that certain sections may be left intentionally blank if they are not necessary in the given context.

```cpp
void List::push_front (int val) {
  auto tmp = std::make_unique<Node>(val); // To construct a new node
  if (head) {
    tmp->next = std::move(head);
    head = std::move(tmp);

  }
  else {
    head = std::move(tmp);
  }
}
```

D.
```cpp
std::vector<int> work(const std::vector<int> &v, int k) {
    std::vector<int> result(v.size());
    std::copy_if(v.begin(), v.end(), result.begin(), [k](int x) { return x < k; });
    return result;
}
```

2. **(15 points) The "is-a" relationship**
Public inheritance models the "is-a" relationship. Explain your understanding on this. Give some good and bad examples.

Solution: Public inheritance means "is-a". Everything that applies to base classes must also apply to derived classes, because every derived class object is a base class object. Anywhere an object of type `Base` can be used, an object of type `Derived` can be used just as well.

Good example: A student is a person, a rectangle is a shape, etc.

Bad example: A square is a rectangle, but not everything applicable to a rectangle can be applied to a square.

总结表格

方法	示例	适用场景
构造函数指定值	`vector<int> vec(5, 42);`	所有元素相同
默认初始化（0）	`vector<int> vec(5);`	数值类型初始化为0
列表初始化	`vector<int> vec{1, 2, 3};`	显式指定每个元素
逐个赋值	`vec[0] = 10;`	修改特定元素
`std::fill`	`fill(vec.begin(), vec.end(), 7);`	批量填充相同值
`std::generate`	`generate(vec.begin(), vec.end(), gen);`	动态生成元素值
从其他容器复制	`vector<int> vec(arr, arr+3);`	基于已有数据初始化

Compiler's diagnostics:

```
tool.cpp: In function 'void runTool()':
tool.cpp:13:12: error: use of deleted function 'std::unique_ptr<_Tp, _Dp>::unique_ptr(cons
t std::unique_ptr<_Tp, _Dp>&) [with _Tp = Action; _Dp = std::default_delete<Action>]'
   13 |   runAction(action);
      |   ~~~~~~~~~^~~~~~~~
In file included from /usr/include/c++/13/memory:78,
                 from tool.cpp:1:
/usr/include/c++/13/bits/unique_ptr.h:522:7: note: declared here
  522 |       unique_ptr(const unique_ptr&) = delete;
      |       ^~~~~~~~~~
tool.cpp:7:40: note:   initializing argument 1 of 'void runAction(std::unique_ptr<Action>)'
    7 | void runAction(std::unique_ptr<Action> action) {
      |                ~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~
```

The compiler says that the code uses a deleted function. What function is it? Why is it used by the code? How will you fix the error **if you can only modify the code in `runTool`**?

Solution: The deleted function this code uses is the copy constructor of `std::unique_ptr`. It is used because on line 13 `action` is an lvalue and is passed by value.
To fix the error, we may pass `std::move(action)` as the argument, or we may just write `runAction(std::make_unique<Action>(/* ... */))` directly.

```cpp
#include <algorithm>

std::vector<int> vec(5);
int counter = 1;
std::generate(vec.begin(), vec.end(),
[&counter]() { return counter++; });
// 结果为 {1, 2, 3, 4, 5}
```

```cpp
int arr[] = {1, 2, 3};
std::vector<int> vec(std::begin(arr), std::end
(arr)); // 复制数组内容

std::vector<int> vec(3); // 初始化为 {0, 0, 0}
vec[0] = 10;             // 修改第一个元素为 10
vec.at(1) = 20;          // 修改第二个元素为 20
 (带边界检查)
for (int& x : vec) x *= 2; // 遍历并修改每个元素
 (变为 {20, 40, 0})
```

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>      // 用于partition和next_permutation
#include <numeric>        // 用于iota
#include <iterator>

void dropDuplicates(std::vector<std::string>& strings) {
  std::sort(strings.begin(), strings.end());
  auto last = std::unique(strings.begin(), strings.end());
  strings.erase(last, strings.end());
}

auto partitionByLength(std::vector<std::string>& strings, std::size_t k) {
  return std::partition(strings.begin(), strings.end(),
        [k](const std::string& s) { return s.length() <= k; });
}

void generatePermutations(int n, std::ostream& os) {
  std::vector<int> numbers(n);
  std::iota(numbers.begin(), numbers.end(), 1);
  do {
    // TODO: Print the numbers, separated by a space.
    std::for_each(numbers.begin(), numbers.end() - 1, [&os](int num) { os << num << " "; });
    os << numbers.back() << '\n';
  } while (std::next_permutation(numbers.begin(), numbers.end()));
}
```

Below is a C-style function, `mySort`, that sorts `Student` instances based on a specified comparison policy.

```cpp
// C-style C++ function
void mySort(std::vector<Student>::iterator begin, std::vector<Student>::iterator end,
            bool(* policy)(const Student&, const Student&)) {
  for (auto it = begin; it != end - 1; ++it) {

    auto pivot = it;

    // Iterate through the unsorted portion
    for (auto it2 = it + 1; it2 != end; ++it2) {
      if (policy(*it2, *pivot)) {
        // Update pivot
        pivot = it2;
      }
    }

    // Move the pivot to its correct position
    std::swap(*it, *pivot);
  }
}
```

Task: Sort the `students` vector in **descending order** by age ***using the `mySort` function.*** Your solution should include the following:

i. (6') **Definition of the auxiliary comparision function:**

Solution:

```cpp
bool func(const Student &lhs, const Student &rhs) {
  return lhs.getAge() > rhs.getAge();
}
```

Alternatively, you can define it with `>=`:

```cpp
bool func(const Student &lhs, const Student &rhs) {
  return lhs.getAge() >= rhs.getAge();
}
```

You will lose **2 points** if the comparison operator's direction is reversed.

ii. (4') **Usage:** Rewrite the invocation of `std::sort` in (3) using the `mySort` function along with the auxiliary comparison function. Provide the solution in **one line of code**.

Solution:

```cpp
mySort(students.begin(), students.end(), &func);
```

cpp 复制 下载

```cpp
std::vector<std::string> words = {"apple", "banana", "cherry", "date"};
std::sort(words.begin(), words.end(), [](const auto& a, const auto& b) {
    return a.size() < b.size(); // 按长度升序
});
// 结果: "date", "apple", "banana", "cherry"
```

4. 排序部分范围

只对前 3 个元素排序:

cpp 复制 下载

```cpp
std::sort(nums.begin(), nums.begin() + 3);
// 原 {4, 2, 5, 1, 3} → {2, 4, 5, 1, 3}
```

```cpp
class Dynarray {
public:
  Dynarray() = default;

  /* (a) */ explicit Dynarray(std::size_t n) : m_length{n}, m_storage{new int[n]{}} {}

  /* (b) */ Dynarray(std::size_t n, int x) : Dynarray(n) {
    std::fill_n(m_storage, n, x);
  }

  /* (c) */ Dynarray(const int *begin, const int *end) : Dynarray(end - begin) {
    std::copy(begin, end, m_storage);
  }

  /* (d) */ Dynarray(const Dynarray &other);

  Dynarray(Dynarray &&other) noexcept
      : m_length{other.m_length}, m_storage{other.m_storage} {
    other.m_length = 0;
    other.m_storage = nullptr;
  }

  ~Dynarray() { delete[] m_storage; }

  void swap(Dynarray &other) noexcept {
    std::swap(m_length, other.m_length);
    std::swap(m_storage, other.m_storage);
  }

  /* (e) */ Dynarray &operator=(Dynarray other) noexcept {
    swap(other);
    return *this;
  }

  // other members ...

private:
  std::size_t m_length{0};
  int *m_storage{nullptr};
};
```

(1) (5') The way that the constructors (b) and (c) are defined is interesting: By writing `Dynarray(n)` and `Dynarray(end - begin)`, they *delegate* part of the initialization work to the constructor (a). Such constructors are called *delegating constructors*.

Can you define the constructor (d) as a *delegating constructor* too? Fill in the blank below.

```cpp
Dynarray::Dynarray(const Dynarray &other)
    : Dynarray(__other.m_storage, other.m_storage + other.m_length__) {}
```

(2) (5') What kind of assignment operator is (e)? Why?

Solution: It is both a copy assignment operator and a move assignment operator.
When assigning a `Dynarray` object with an expression *`expr`*, the parameter `other` will be initialized from *`expr`*, which performs copy-construction if *`expr`* is an lvalue and move-construction if it is an rvalue.

`std::count_if` is a standard library function that accepts **a sequence** and **a unary predicate**, and returns the number of elements in the sequence for which the predicate returns `true`. Use `std::count_if` to rewrite the code above.

```cpp
int cnt = std::count_if(
    ____strings.begin()____,
    ____strings.end()____,
    ____[k](const std::string &s) { return s.size() < k; }____
);
```

数组传参：全部为*T类型*。

Even if you declare the parameter as an array (either T a[N] or T a[]), its type is still a pointer T : You are allowed to pass anything of type T* to it.

```cpp
void fun(int *a);
void fun(int a[]);
void fun(int a[10]);
void fun(int a[2]);//声明的数组大小并没有实际的意义和约束
```

多维数组传参及辨析

int (*a)[N]:一个指针，指向一个类型为int[N]的一维数组

int *a[N]：一个一维数组，含有N个int *

二维数组int[N][M]传参方法：

```cpp
void fun(int (*a)[N]);
void fun(int a[][N]);
void fun(int a[2][N]);
void fun(int a[10][N]);//第一维的数值仍没有明确限制意义
//但第二维必须为N，表示了指向的一维数组的具体类型
```

```cpp
4. void fun(int **a) : A pointer to int * . No.
5. void fun(int *a[]) : Same as 4.即：这是一个指向a[]的指针，
//又因为数组自动退化成指针，所以是一个指向int*的指针。
6. void fun(int *a[N]) : Same as 4.
```

(5) (5') [C++] Let `a` be of type `int [9][10]`. Select the pieces of code that makes `foo(a)` compile.

A. `void foo(int **a);`
B. `void foo(int (*a)[9]);`
C. `void foo(int (&a)[9][10]);`
D. `void foo(int (&a)[10][9]);`

(6) (5') [C++] We want to use a nested `vector` to represent a matrix. Which of the following statements is/are true?

A. `std::vector<std::vector<double>>` is a valid type.
B. `std::vector<std::vector<double>> matrix(n, m);`
`matrix` is initialized to be with `n` rows and `m` columns. That is, `matrix` is a vector that contains `n` elements, each of which is a vector containing `m` `double`s.
C. `std::vector matrix(n, std::vector(m, 0.0));`
The type of `matrix` is deduced to be `std::vector<std::vector<double>>`.
D. `std::vector matrix(n, std::vector(m, 0.0));`
`matrix` is initialized to be with `n` rows and `m` columns, with each element initialized to `0.0`.

(7) (5') [C++] Select the pieces of code in which the following range-based for loop can be used to traverse `a`.

```cpp
for (const auto &x : a)
  do_something_with(x);
```

A. `int a[100]{};`
B. `int *a = new int[100]{};`
C. `std::vector<std::string> a(10, "hello");`
D. `std::string a = "world";`